Informe Laboratorio 1

Sección 1

Oseas Poveda Huerta
e-mail: oseas@mail.udp.cl
Ingeniería Civil Informática y Telecomunicaciones
Universidad Diego Portales
Github: 0s3a5

Agosto de 2026

Índice

1. Descripción **2**

2. Actividades **2**
2.1. Algoritmo de cifrado 2
2.2. Modo stealth 2
2.3. MitM 3

3. Desarrollo de Actividades **5**
3.1. Actividad 1: Algoritmo de cifrado 5
3.2. Actividad 2: Modo Stealth 6
3.3. Actividad 3: MitM 10

1. Descripción

1. Usted empieza a trabajar en una empresa tecnológica que se jacta de poseer sistemas que permiten identificar filtraciones de información a través de Deep Packet Inspection (DPI). A usted le han encomendado auditar si efectivamente estos sistemas son capaces de detectar las filtraciones a través de tráfico de red. Debido a que el programa ping es ampliamente utilizado desde dentro y hacia fuera de la empresa, su tarea será crear un software que permita replicar tráfico generado por el programa ping con su configuración por defecto, pero con fragmentos de información confidencial. Recuerde que al comparar tráfico real con el generado no debe gatillar alarmas. De todas formas, deberá hacer una prueba de concepto, en la cual se demuestre que al conocer el algoritmo, será fácil determinar el mensaje en claro. Para los pasos 1,2,3 indicar el texto entregado a ChatGPT y validar si el código resultante cumple con lo requerido.

2. Actividades

2.1. Algoritmo de cifrado

1. Generar un programa, en python3 utilizando chatGPT, que permita cifrar texto utilizando el algoritmo Cesar. Como parámetros de su programa deberá ingresar el string a cifrar y luego el corrimiento.

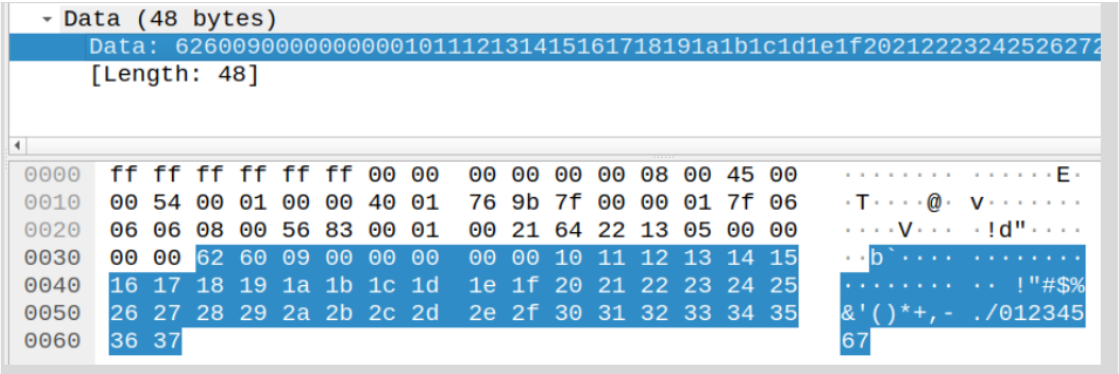
```
~/Desktop sudo python3 cesar.py "criptografia y seguridad en redes" 9
larycxpajorj h bnpdarmjm nw anmnb
```

2.2. Modo stealth

1. Generar un programa, en python3 utilizando ChatGPT, que permita enviar los caracteres del string (el del paso 1) en varios paquetes ICMP request (un caracter por paquete en el campo data de ICMP) para de esta forma no gatillar sospechas sobre la filtración de datos. Deberá mostrar los campos de un ping real previo y posterior al suyo y demostrar que su tráfico consideró todos los aspectos para pasar desapercibido.

```
~/Desktop sudo python3 pingv4.py "larycxpajorj h bnpdarmjm nw anmnb"
.
Sent 1 packets.
.
Sent 1 packets.
.
Sent 1 packets.
.
Sent 1 packets.
```

El último carácter del mensaje se transmite como una b.



2.3. MitM

1. Generar un programa, en python3 utilizando ChatGPT, que permita obtener el mensaje transmitido en el paso 2. Como no se sabe cual es el corrimiento utilizado, genere todas las combinaciones posibles e imprímalas, indicando en verde la opción más probable de ser el mensaje en claro.

```
~/Desktop sudo python3 readv2.py cesar.pcapng
0         larycxpajorj h bnpdarmjm nw anmnb
1         kzqxbwozinqi g amoczqlil mv zmlma
2         jypwavnyhmph f zlnbypkhk lu ylklz
3         ixovzumxglog e ykmaxojgj kt xkjky
4         hwnuytlwfknf d xjlzwnifi js wjijx
5         gvmtxskvejme c wikyvmheh ir vihiw
6         fulswrjudild b vhjxulgdg hq uhghv
7         etkrvqitchkc a ugiwtkfcf gp tgfgu
8         dsjquphsbgjb z tfhvsjebe fo sfeft
9         criptografia y seguridad en redes
10        bqhosnfqzehz x rdftqhczc dm qdcdr
11        apgnrmepydgy w qcespgbyb cl pcbcq
12        zofmqldoxcfx v pbdrofaxa bk obabp
13        ynelpkcnwbew u oacqnezwz aj nazao
14        xmdkojbmvadv t nzbpmdyvy zi mzyzn
15        wlcjnialuzcu s myaolcxux yh lyxym
16        vkbimhzktybt r lxznkbwtw xg kxwxl
17        ujahlgyjsxas q kwymjavsv wf jwvwk
18        tizgkfxirwzr p jvxlizuru ve ivuvj
19        shyfjewhqvyq o iuwkhytqt ud hutui
20        rgxeidvgpuxp n htvjgxsps tc gtsth
21        qfwdhcufotwo m gsuifwror sb fsrsg
22        pevcgbtensvn l frthevqnq ra erqrf
23        odubfasdmrum k eqsgdupmp qz dqpqe
24        nctaezrclqtl j dprfctolo py cpopd
25        mbszdyqbkpsk i coqebsnkn ox bonoc
```

Finalmente, deberá indicar 4 issues que haya tenido al lidiar con ChatGPT, netamente para reflejar cuál fue su experiencia al trabajar con esta tecnología.

3. Desarrollo de Actividades

La actividad se desarrollo siguiendo las instrucciones de la guía y en cumplimiento de lo indicado por la rubrica, todo lo relacionado a las IA y los prompts también se indicaran acá, cabe destacar que los promps puestos son el resultado final y el ultimo con el cual se realizo esta actividad.

3.1. Actividad 1: Algoritmo de cifrado

En la primera actividad se nos pedía que le pidiera a Chatgpt que generara un programa que realice un cifrado mediante algoritmo cesar, lo cual es lo mismo que un rot solo que en este caso se nos pida que sea con un desplazamiento de 9 letras.

Prompt ingresado a ChatGPT: *"Escribe un script en Python 3 usando la librería sys para recibir exactamente dos argumentos por la terminal: un string a cifrar y el número de corrimiento. Implementa el algoritmo César manteniendo los espacios intactos."*

Una vez la IA entrega el código se procede a ejecutarlo dando el siguiente resultado

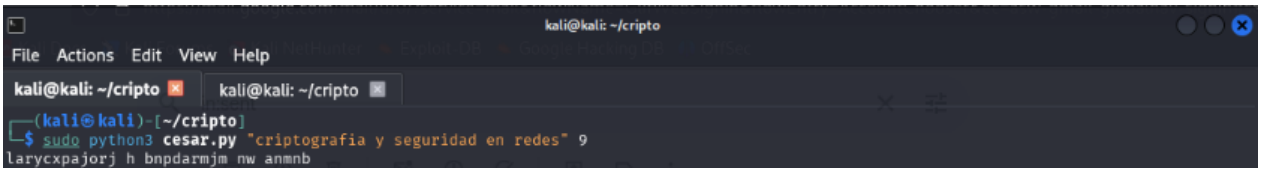


Figura 1: Ejecución de algoritmo cesar

Esto se comprueba que esta correcto al realizar los cálculos manualmente y sobre todo al comprobar lo que dice la guía ya que el mensaje encriptado termina en b.

```
def cifrar_cesar(texto, corrimiento):
resultado = ""
for char in texto:
    if char.isalpha():
        ascii_base = ord('a') if char.islower() else ord('A')
        resultado += chr((ord(char) - ascii_base + corrimiento)
        % 26 + ascii_base)
    else:
        resultado += char
return resultado
```

Dentro del código, lo que mas quiero resaltar es que se reciben 2 cosas, el texto y el corrimiento, y en este código da lo mismo si es una mayúscula o una minúscula. Lo que se hace es pasar el char uno por uno a numero según la tabla ASCII, y luego se le ejecuta el corrimiento deseado, en caso de caer en el else se le asigna como espacio

3.2. Actividad 2: Modo Stealth

Para la siguiente actividad se nos pide que nuevamente le pidamos a la IA un programa para enviar cada uno de los caracteres en diferentes paquetes ICMP, al mismo tiempo para la verificación de esto se utiliza wireshark para la intercepcion de paquetes, por el cual también podemos ver la anormalidad del trafico.

Prompt ingresado a ChatGPT: *"Crea un script en Python 3 usando Scapy que inyecte un mensaje cifrado en tráfico ICMP debe ir a la IP de loopback y mandarse cada 1 segundo manteniendo el envió como si fuera real."*

A continuación se muestra el código completo entregado por la IA, encargado de fragmentar el mensaje cifrado y de inyectar cada carácter dentro de un paquete ICMP distinto, replicando la estructura de un ping real generado por Linux (Kali):

```
import sys
import time
import struct
from scapy.all import IP, ICMP, send

if len(sys.argv) != 2:
    print("Uso: python3 pingv4.py \"texto_cifrado\"")
    sys.exit(1)

texto_cifrado = sys.argv[1]
ip_dest = "127.0.0.1"
ip_id = 1000
seq_num = 1
icmp_id = 1234

for char in texto_cifrado:
    timestamp_real = struct.pack("<Q", int(time.time()))
    char_byte = char.encode('utf-8')
    timestamp_2_bytes = timestamp_real
    timestamp real
    cinco_ceros = b'\x00\x00\x00\x00\x00'
    payload_padding = bytes(range(0x10, 0x38))

    # Ensamblamos los 48 bytes exactos del payload
    data = char_byte + timestamp_2_bytes + cinco_ceros + payload_padding

    # Al no declarar el checksum manual, Scapy lo calcula de manera coherente
    pkt = IP(dst=ip_dest, id=ip_id) / ICMP(id=icmp_id, seq=seq_num) / data
```

```
send(pkt, verbose=False)
print(f"Sent 1 packets.")

# Incrementamos los contadores para mantener la coherencia de
secuencia e identificacion
ip_id += 1
seq_num += 1

# Mantiene la ejecucion ciclica para no saturar y pasar desapercibido
time.sleep(1)
```

Dentro del código se puede identificar que el mensaje cifrado se recibe como argumento y se recorre carácter por carácter, generando un paquete ICMP independiente por cada uno. La dirección de destino corresponde a la IP de loopback (`127.0.0.1`), observable posteriormente mediante Wireshark.

- **IP Identification:** en un ping real de Linux, el campo `id` de la cabecera IP se incrementa de forma secuencial en cada paquete enviado. En el script esto se replica explícitamente con la variable `ip_id`, que parte en 1000 y se incrementa en 1 tras cada envío (`ip_id += 1`), imitando el comportamiento observado en el ping de referencia de la Figura.
- **Sequence Number (ICMP):** de igual forma que un ping real incrementa su sequencia en cada solicitud, el script mantiene la variable `seq_num` que comienza en 1 y aumenta en 1 por cada paquete (`seq_num += 1`), coincidiendo con el patrón ascendente que se observa al comparar ambas capturas en Wireshark.
- **ICMP Identifier:** en un ping real este valor permanece constante durante toda la sesión de ping, ya que identifica al proceso que generó la solicitud. El script fija (`ICMP_ID = 1234`) de forma estática para todos los paquetes enviados, tal como se observa que ocurre en el ping real de referencia.
- **Checksum:** en las primeras versiones del script el checksum se calculaba manualmente, lo que generaba inconsistencias visibles en Wireshark. En la versión final, al no declarar el checksum de forma manual y dejar que Scapy lo calcule automáticamente al construir el paquete.
- **Timestamp:** se conserva parte del timestamp real del sistema, tomando los bytes correspondientes a la porción menos significativa, de forma que el valor cambia de manera progresiva entre paquete y paquete, tal como sucede en el timestamp de un ping real, evitando que quede estático.

Luego tenemos el envió al ejecutar el programa.

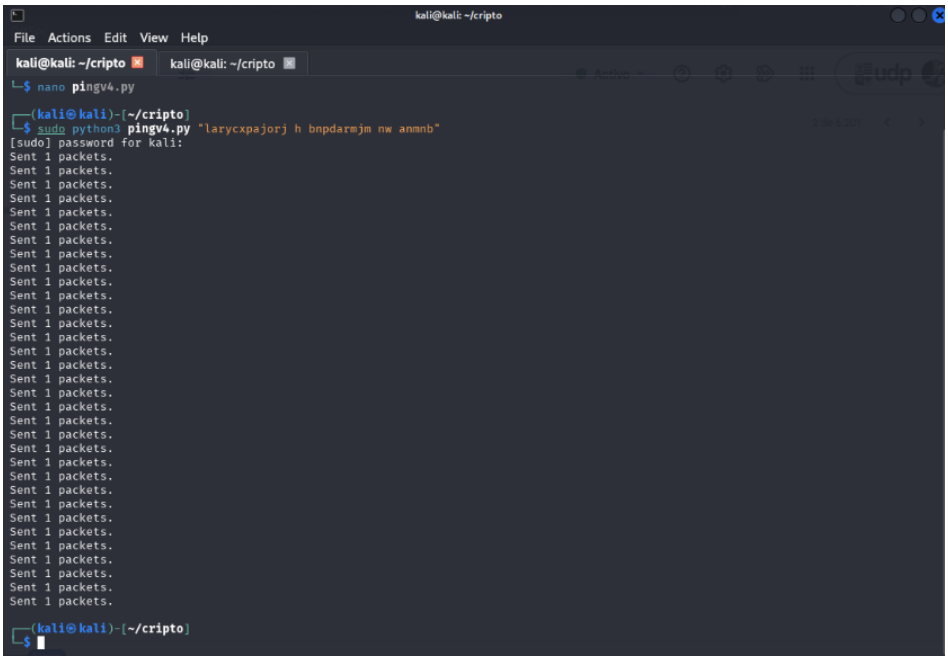


Figura 2: Envió de paquetes

Antes de ejecutar el programa se debe abrir el wireshark y se debe de poner en loopback, una vez empiece a buscar en la red se le añade el filtro ICMP.

Luego para se ejecuta el segundo programa con:

```
sudo python3 pingv4.py "larycxpajorj h bnpdarmjm nw anmnb"
```

Como podemos evidenciar se ve el envió de paquetes, estos se han enviado cada 1 segundo para una mejor simulación, ademas de que parezca trafico normal. Con esto en wireshark al identificar los paquetes no parecen diferentes, ni tampoco generan alerta al compararlos con el ping real de referencia, con esto dicho podemos pasar al análisis de wireshark.

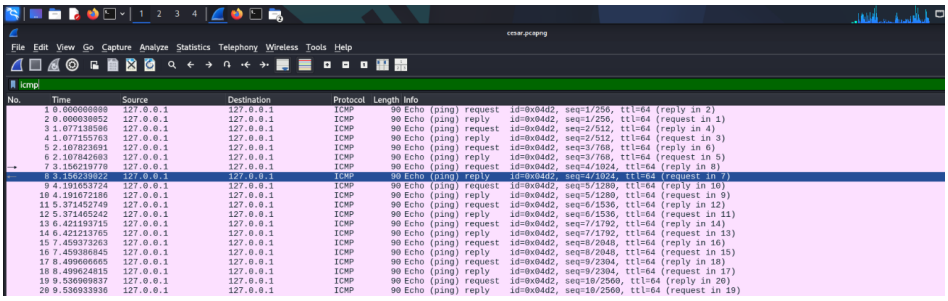


Figura 3: Trafico de paquetes

En esta captura que se obtiene de wireshark,en ella podemos identificar que los paquetes se envían correctamente a la dirección IP deseada, ademas de que podemos ver como es que

entre cada envió (request) se realiza cada 1 segundo, logrando que se envié como si fuera un trafico normal, luego podemos observar como es el desglose de uno de los paquetes, con esto se evidencia fielmente que no se modifica el timestamp.

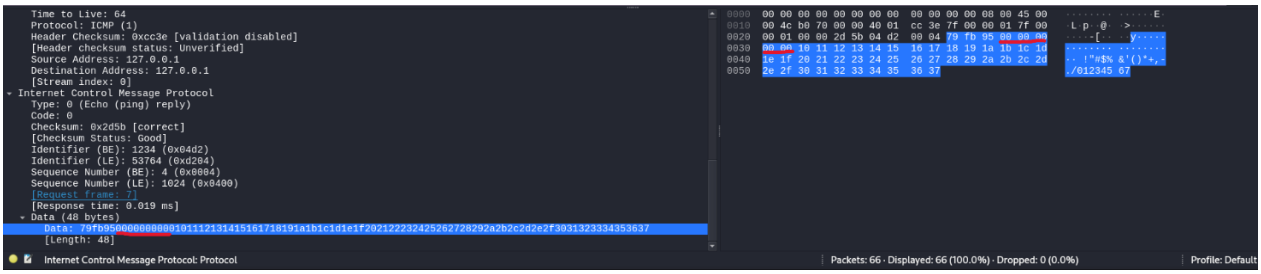


Figura 4: Muestra de estructura

Lo que quiero evidenciar con esta imagen es que tal como se pide en la guía, se puede ver el el payload de 48 bytes de un ping de linux (en este caso de Kali), esto genera que se divida en cuatro categorías, correspondientes a las cuatro secciones definidas dentro del código:

1. **Byte 0 Carácter enviado:** corresponde a `char_byte`, el carácter cifrado que se está transmitiendo en ese paquete específico (por ejemplo, la letra `b` en el último paquete del mensaje).

2. **Bytes 1 y 2 Timestamp:** corresponden a `timestamp_2_bytes`, una porción del timestamp real del sistema al momento del envío, lo que otorga variabilidad coherente entre paquetes.

3. **Bytes 3 al 7 Cinco ceros:** corresponden a `cinco_ceros`, exigidos explícitamente por la rúbrica y presentes en la estructura por defecto del payload de ping en Linux, el mas importante porque después de este viene la secuencia y esta secuencia es la que la guía nos dice que debemos de ver y no puede modificar o dejar sin secuencia o estática.

4. **Bytes 8 al 47 Padding hexadecimal** corresponden a payload padding, el abecedario en hexadecimal característico del payload por defecto de ping en Linux, que se mantiene idéntico al de un ping real y es la muestra de que el resto del mensaje no es estático y que verdaderamente se modifica solo lo necesario para que pase desapercibido.

3.3. Actividad 3: MitM

Prompt ingresado a ChatGPT: *"Desarrolla un script en Python 3 que reciba un archivo .pcapng por la terminal y usa scapy para leerlo, saca los bytes útiles del ICMP y reconstruir el string, de modo iterativo identifica el mensaje correcto y muestralo en color verde."*

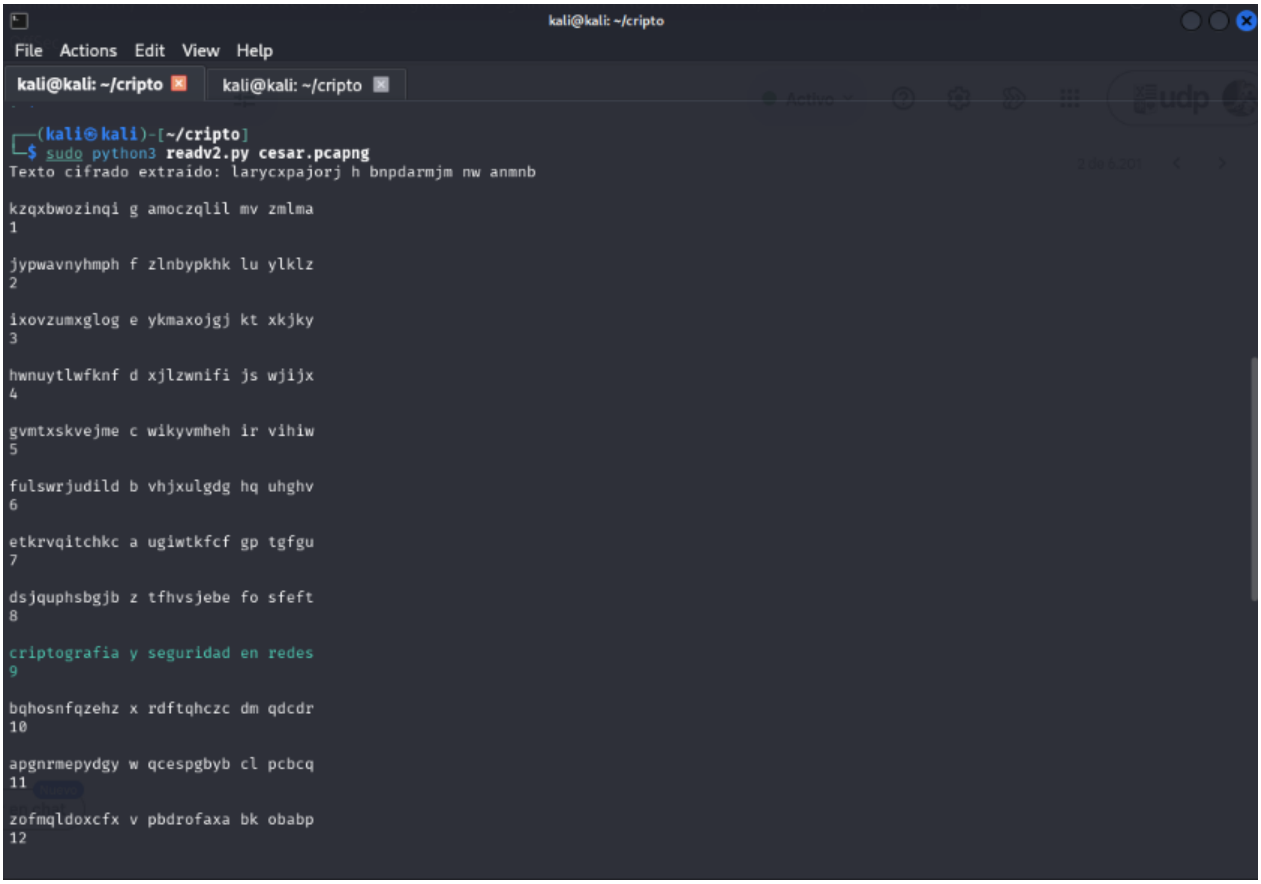


Figura 5: Desencriptación

Para poder ejecutar este ultimo programa se debe de tener la captura anterior terminada y guardad en la misma carpeta, y una vez teniendo todo esto listo se ejecuta la siguiente linea en la terminal para poder descifrar el mensaje.

```
sudo python3 readv2.py cesar.pcapng
```

En esta imagen (aunque no se alcanza a ver completo) se realizan las 26 combinaciones posibles(que son los 26 corrimientos), para identificar cual es, esto lo hace el programa con una palabra clave la cual es criptografía, con esta palabra identificada en la linea 9 (ya que este es el corrimiento que se le da) se pone de color verde para así dar por finalizado el laboratorio

Conclusiones y comentarios (Issues encontrados)

A lo largo de este laboratorio, lidiar con ChatGPT fue revelador. Lejos de ser un ente omnisciente, requiere microgestión absoluta y un pensamiento sumamente crítico si se busca precisión técnica en redes. Aquí detallo los 4 issues principales que reflejan la experiencia:

1. **Issue 1: Destrucción del Padding ICMP.** Si le pides a la IA que inyecte un byte, borra todo el resto del payload estándar de linux por lo cual se debe revisar bien primeramente si es que los bytes están correctos y al no estarlos se le pide a GPT que lo realice de manera correcta.

2. **Issue 2: El Checksum.** Gpt hacia los cálculos de IP y de ICMP a mano en el código y en wireshark eso generaba errores, por lo cual se debe definir bien que debe ocupar la lógica de scapy y no de manera manual para que el trafico pasara desapercibido.

3. **Issue 3: Timestamp Incoherente.** Al analizar nuevamente en wireshark ya no ocurría el error de que el payload desaparecia, sino que ahora lo que aparecía eran errores ya que no había una estructura, por lo cual ahora se especifica que se debe de hacer en struct y que debe de mantener coherencia para no ser detectado.

4. **Issue 4:No se ve verde.** El prompt pedía pintar en verde el mensaje en claro, pero la IA no lograba mostrar correctamente el color al no detectar nada para pintar, por lo cual se le añade una verificación para que cuando se tuviera una palabra clave este se pudiera mostrar de color verde como lo es en nuestro caso, la palabra criptografía.

Al terminar el laboratorio se puede decir que la IA es un buena para realizar actividades y que optimiza bastante el tiempo para los resultados, pero siempre se deben de corroborar los resultados ademas de que se tienen que realizar las pruebas de manera correcta para identificar cuales son las falencias que la IA genera, el ejemplo mas claro es lo de los timestamp ya que si bien la IA logro hacerlo, hay pequeños detalles que se tienen que arreglar y sin un conocimiento mínimo de estos no se puede identificar el error.